UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

EVALUACION	PARCIAL	**GRUPO**	M4A	**FECHA**	18/09/2025
MATERIA	DISEÑO DE APLICACIONES				
CARRERA	ANALISTA EN TECNOLOGIAS DE INFORMACIÓN / ANALISTA PROGRAMADOR				
CONDICIONES	**- Puntos: 20** **- Duración 2 HORAS** **- Sin material** **- Debe entregar esta letra junto a la solución**				

Ejercicio 1 (8 puntos)

Un conocido grupo empresario propietario de varias cocheras de la ciudad, decide embarcarse en un proyecto que automatice varios procesos vinculados a la gestión de las playas de estacionamiento de su propiedad.

Como experiencia piloto se han propuesto iniciar con una primera playa de estacionamiento, aunque planifican incorporar una nueva playa cada seis meses una vez que esté probada la solución.

De cada playa de estacionamiento se sabe su dirección y su nombre. Cada playa tiene cocheras, de las cuales se sabe su nivel/piso, su número y los tipos de vehículos admitidos. De cada tipo de vehículo se sabe su nombre, alto máximo, ancho máximo y largo máximo, todo medido en centímetros.

De cada vehículo se sabe su dominio y tipo de vehículo. Cada tipo de vehículo tiene un precio a pagar por hora de estadía.

Se pide: realizar el diagrama conceptual de dominio del caso anteriormente descripto incluyendo entidades, sus relaciones y multiplicidad.

Ejercicio 2 (12 puntos)

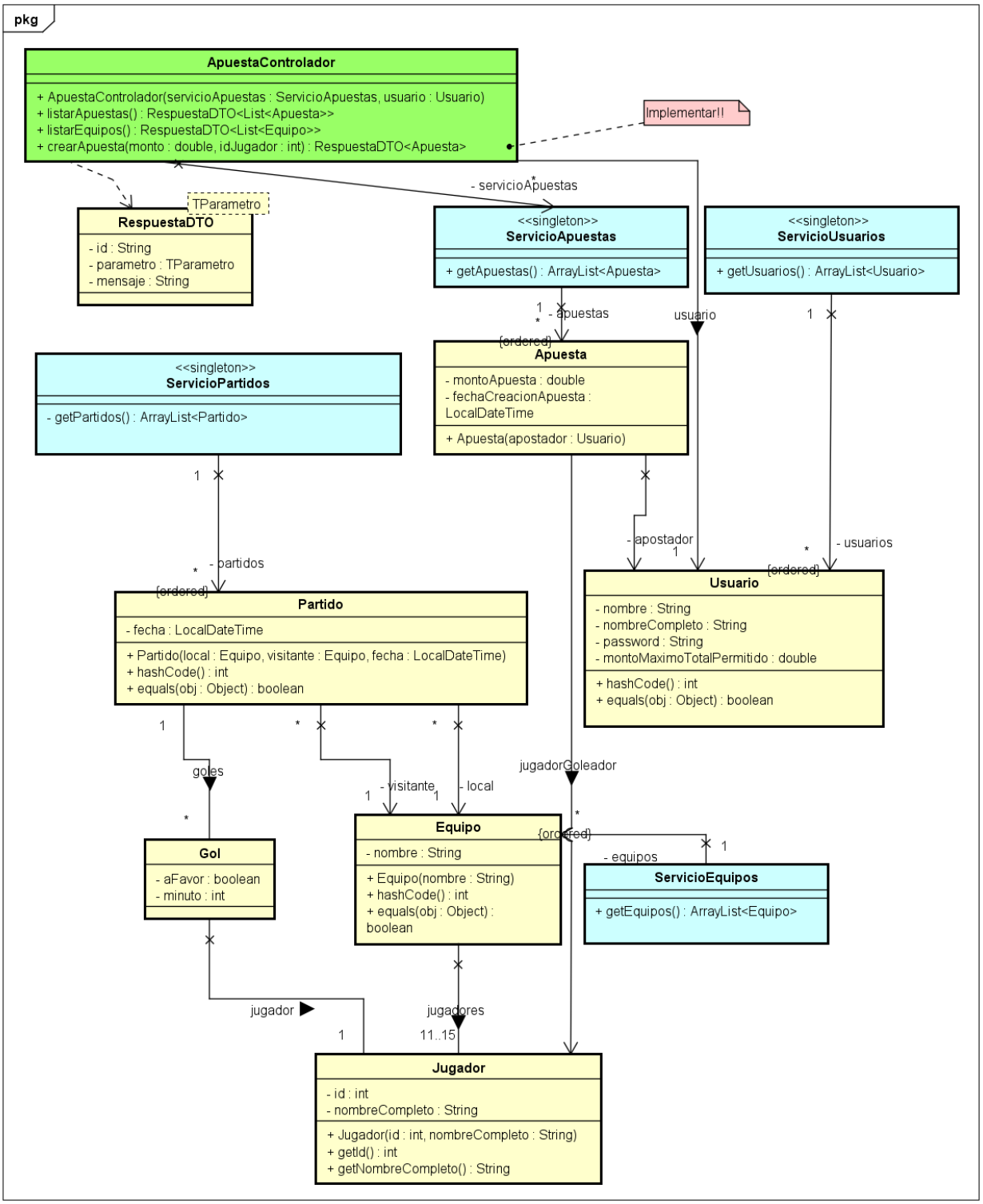
La liga universitaria de futbol decide implementar un prototipo de sistema de apuestas que permita recaudar fondos. Para la primera versión se acordó que los usuarios podrán apostar un monto máximo que operará de limite por cada torneo y cada usuario definirá el mismo. Inicialmente se permitirá apostar a quien será el goleador del torneo por un monto determinado, aunque a futuro es posible que se admitan otros tipos de apuestas. De cada apuesta se sabe el usuario que apuesta, el monto apostado, el partido y el jugador al cual apuesta que será goleador.

El siguiente es el endpoint definido en el controlador:

```
@RestController
@RequestMapping("/apuestas")
public class ApuestaControlador {

    @PostMapping("/crear")
    public RespuestaDTO<Apuesta> crearApuesta(@RequestParam double monto, @RequestParam int idJugador) {
//Implementar!!}
```

Dado el siguiente diagrama de diseño de solución:



Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel 902 15 05 Fax 908 13 70
www.ort.edu.uy

CU: Alta de apuestas a goleador del torneo

Precondición: el usuario se encuentra autenticado.

Curso normal:

Paso #	Usuario	Sistema
1		Verifica que aún queden partidos sin jugar y muestra en pantalla los equipos.
2	Selecciona un equipo	Lista el nombre de los jugadores del equipo seleccionado
3	Selecciona el jugador por el cual desea realizar la apuesta a goleador del torneo e ingresa el monto de la apuesta	Valida: • que el monto sea positivo • se eligió un jugador existente • que el total de apuestas del usuario no supere el monto máximo establecido por dicho usuario [usuario.getMontoMaximoTotalPermitido()]. Se registra la apuesta estableciendo la fecha en que se dio de alta la misma [apuesta.setFechaCreacionApuesta(LocalDateTime.now())]. Muestra los detalles de la apuesta realizada según el siguiente formato: "Apuesta creada exitosamente a favor de goleador: [apuesta.getJugador().getNombreCompleto()] – Monto apostado: [apuesta.getMontoApuesta()]"

Cursos Alternativos:
1. No hay partidos por jugar, el sistema muestra el mensaje: "No se pueden recibir apuestas". Fin del CU.
3. El monto de apuesta es menor o igual a 0 y/ó el monto total de apuestas es mayor: el sistema muestra el mensaje "No se puedo registrar la apuesta". Se ejecuta el paso #3 del curso normal.
El jugador es nulo o no existe: el sistema muestra el mensaje "No se puedo registrar la apuesta.". Se ejecuta el paso #3 del curso normal.

Se pide: implementar en java el caso de uso descripto (no debe implementar la vista).

Consideraciones:
- Tener en cuenta que todos los métodos que aparecen en el diagrama de clases ya se encuentran implementados y algunos podrían ser útiles para su solución. **Utilizar estrictamente el diagrama de clases dado en la letra como base para su solución. Se podrán agregar nuevas clases, métodos y relaciones.**
- Debe implementar **ApuestaControlador.crearApuesta**(monto, idJugador):RespuestaDTO<Apuesta>
- No es necesario implementar getters/setters.
- No puede haber lógica/código duplicado.